CSCI3360 Introduction to Computational Complexity (2026 Fall)

Xiao Liang

https://xiao-liang.github.io

Department of Computer Science and Engineering
The Chinese University of Hong Kong

(Updated on September 6, 2026)

Disclaimer: These lecture notes originate from the 2026-Fall offering of the course. They have been lightly edited for accuracy and completeness, and may still contain typographical or other errors.

Contents

Chapter 1
Models of Computation

Computational complexity asks not only whether a problem can be solved, but also how many computational resources are required to solve it. Before we can measure resources such as time and memory, however, we must specify what counts as a computation. In other words, we need a mathematical model of an algorithm.

We begin by recalling two models from formal-language theory: finite automata and pushdown automata, which were already covered in **CSCI3130 Formal Languages and Automata Theory**. These models are useful precisely because their memory is restricted, but the same restrictions prevent them from expressing general computation. We then introduce the Turing machine, work through algorithms for recognizing palindromes and a non-context-free language, and examine the robustness of the model. Finally, we introduce universal Turing machines and discuss the Church–Turing thesis and its complexity-theoretic extension.

To study the complexity of computation, we must first define a model of computation. The model must be precise enough for mathematical proofs and expressive enough to represent general algorithms.

1.1 Before Turing Machines: Restricted Models of Computation

Nothing from Section 1.1 will be tested on quizzes or exams!

1.1.1 Finite Automata

Recall that a deterministic finite automaton has a finite set of states and reads its input from left to right. At every step, its next state is determined by its current state and the next input symbol.

Definition 1.1.1 (Deterministic Finite Automaton)**.** *A* deterministic finite automaton*, or DFA, is a tuple*

$$A = (Q, \Sigma, \delta, q_0, F),$$

where:

- Q *is a finite set of states;*
- Σ *is a finite input alphabet;*
- $\delta : Q \times \Sigma \to Q$ *is the transition function;*
- $q_0 \in Q$ *is the start state; and*
- $F \subseteq Q$ *is the set of accepting states.*

The automaton reads its input only once, from left to right. It accepts an input $x \in \Sigma^*$ if the state reached after reading all of x belongs to F. ◊

The current state of a DFA contains all the information that the machine retains about the portion of the input that it has already read. Since Q is finite, a DFA has only a constant amount of memory, independent of the input length.

Example 1.1.1 (A Language Recognized by a DFA)**.** *Consider the language*

$$L_{\text{even}} \coloneqq \{x \in \{0,1\}^* : x \text{ contains an even number of } 1\text{'s}\}.$$

A two-state DFA recognizes this language. One state records that the number of 1*'s seen so far is even, and the other records that it is odd. Reading a* 1 *switches states, while reading a* 0 *leaves the state unchanged. This is illustrated in Figure 1.1.*

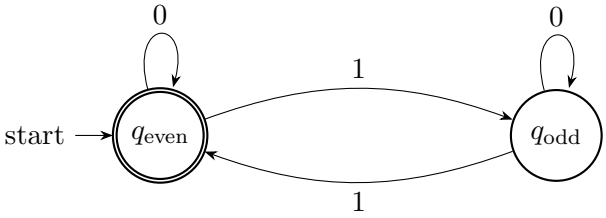

Figure 1.1: A DFA recognizing binary strings containing an even number of 1's.

The boundary of finite-state computation

Finite automata recognize exactly the *regular languages*. They are well suited to problems in which only a bounded amount of historical information is needed.

For example, finite automata are used in the lexical-analysis stage of a compiler. A lexer may recognize identifiers, integer literals, punctuation, and keywords using regular expressions and finite-state machinery.

The limitation is that a DFA cannot store an unbounded counter. Consequently, it cannot recognize the language

$$L_{\text{eq}} \coloneqq \{0^n 1^n : n \geq 0\}. \tag{1.1}$$

To recognize L_{eq}, a machine must compare an arbitrarily large number of 0's with an arbitrarily large number of 1's.

A finite set of states cannot remember the required number for all possible input lengths.

1.1.2 Pushdown Automata

A pushdown automaton extends a finite automaton by providing it with a stack. The stack can grow with the input, so a pushdown automaton can remember an unbounded amount of information. However, that information can be accessed only in last-in, first-out order.

Definition 1.1.2 (Deterministic Pushdown Automaton)**.** *A* deterministic pushdown automaton *(DPDA) is a* 6*-tuple*

$$M = (Q, \Sigma, \Gamma, \delta, q_0, F),$$

where:

1. *Q is a finite set of states;*
2. *Σ is the finite input alphabet;*

3. Γ *is the finite stack alphabet;*

4. $\delta\colon Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \rightharpoonup Q \times \Gamma_\varepsilon$ *is the transition function; Here,*

$$\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\} \qquad \textit{and} \qquad \Gamma_\varepsilon = \Gamma \cup \{\varepsilon\},$$

where ε denotes the empty string.

5. $q_0 \in Q$ *is the start state; and*

6. $F \subseteq Q$ *is the set of accept states.*

The automaton reads its input only once, from left to right. It accepts an input $x \in \Sigma^*$ if the state reached after reading all of x belongs to F.

$\Diamond$

Remark 1.1.1 (On Nondeterminism)**.** *There is a more powerful version called a* nondeterministic pushdown automaton *(NPDA), in which the transition function is allowed to return a set of possible next states and stack symbols. Deterministic and nondeterministic pushdown automata do not have the same expressive power. Every language recognized by a DPDA can be recognized by an NPDA, but the converse does not hold. More precisely,*

$$\text{DCFL} \subsetneq \text{CFL}.$$

Thus, NPDAs recognize all context-free languages, whereas DPDAs recognize only the proper subclass of deterministic context-free languages.

Think of the stack as restricted working memory. The machine can access the most recently stored symbol, but it cannot directly inspect a symbol buried in the middle of the stack.

Example 1.1.2 (Recognizing 0^n1^n with a Stack)**.** *A DPDA can recognize L_{eq} from Equation (1.1) as follows:*

1. *For each* 0 *in the first part of the input, push one marker onto the stack.*

2. *When the first* 1 *is encountered, begin popping one marker for each* 1*.*

3. *Accept if the input ends exactly when the stack returns to its initial marker.*

For the input 000111*, the stack first grows by three markers and then shrinks by three markers. For* 00111*, the machine attempts to pop a third marker after only two have been stored, so it rejects.*

Nondeterministic pushdown automata recognize exactly the *context-free languages*. In compiler design, they provide the mathematical basis for parsing nested syntactic structures, such as balanced parentheses, arithmetic expressions, and recursively nested blocks.

The boundary of stack-based computation

Although a stack is more powerful than finite-state memory, one stack does not provide general-purpose random-access memory. A standard example beyond the power of pushdown automata is

$$L_{\text{three}} \coloneqq \{0^n1^n2^n : n \geq 0\}. \tag{1.2}$$

A machine recognizing this language must verify that three independently delimited blocks have the same length.

A stack can conveniently match the 0's against the 1's by pushing for the first block and popping for the second. After those comparisons, however, the stored count has been destroyed, leaving no copy with which to check the 2's.

Fact 1.1.1 (Pushdown-Automaton Boundary)**.** *The language L_{three} in Equation (1.2) is not context-free. Therefore, no pushdown automaton recognizes it.*

The statement in Fact 1.1.1 can be proved using the pumping lemma for context-free languages or Ogden's lemma. The important point for us is conceptual: the stack provides unbounded but highly structured memory, and that structure imposes a genuine limit on the languages a PDA can recognize.

Takeaway

Finite automata and pushdown automata remain important because their restrictions make them suitable for particular applications and allow precise classifications of languages. They are not, however, expressive enough to represent every algorithm that we would normally regard as executable by a computer. This motivates the Turing-machine model.

1.2 The Three-Tape Turing Machine

Our next goal is to formalize a general-purpose model of computation. A Turing machine combines finite control with tapes that serve as unbounded memory.

1.2.1 Informal Description

An informal algorithm consists of a fixed collection of mechanical rules that can be applied repeatedly. A typical step may perform the following operations:

- read symbols from the input or working memory;
- write symbols to working memory;
- update a finite control state;
- move to nearby memory locations; and
- halt with an output.

A Turing machine turns this description into a precise mathematical object. Each tape is divided into cells, and each cell stores one symbol from a finite alphabet. A tape head reads the symbol in the current cell and may move one position left or right.

The tapes in our definition are unbounded to the right and have a leftmost cell. This is a mathematical abstraction: an execution lasting T steps can visit at most $T + 1$ cells on any tape, so every finite computation uses only a finite portion of each tape.

1.2.2 Formal Definition

Definition 1.2.1 (Three-Tape Turing Machine)**.** *A* three-tape Turing machine *is a tuple*

$$M = (\Gamma, Q, \delta),$$

with the following components:

- Γ *is a finite set of symbols that M's tape can contain. We call it the alphabet.*

  We assume that it contains a blank symbol $\sqcup$, a left-end marker $\triangleright$, and a set of some characters[1] *(e.g., the English alphabet, or the binary bits $\{0, 1\}$).*

- Q *is a finite set of states containing a start state q_{start} and a halting state q_{halt}.*

- *The transition function is*

$$\delta : Q \times \Gamma^3 \longrightarrow Q \times \Gamma^2 \times \{L, S, R\}^3. \tag{1.3}$$

The first tape is a read-only input tape. The second tape is a read/write work tape, and the third tape is a read/write output tape. $\Diamond$

Let us unpack the transition function in Equation (1.3). Suppose

$$\delta(q, a, b, c) = (q', b', c', d_1, d_2, d_3).$$

This transition instructs the machine to:

1. change its state from q to q';
2. leave the input symbol a unchanged; (recall that the input tape is read-only;)
3. replace the work-tape symbol b by b';
4. replace the output-tape symbol c by c'; and
5. move the three heads according to

$$d_1, d_2, d_3 \in \{L, S, R\},$$

   where L, S, and R mean move left, stay in place, and move right, respectively.

If a head is at the left-end marker and is instructed to move left, it remains in place.

Definition 1.2.2 (Start Configuration)**.** *On input $x = x_1 x_2 \cdots x_n \in \Gamma^*$, the* start configuration *of a three-tape Turing machine is defined as follows:*

- *the input tape contains*

$$\triangleright x_1 x_2 \cdots x_n \sqcup \sqcup \cdots ;$$

- *the work and output tapes each contain*

$$\triangleright \sqcup \sqcup \sqcup \cdots ;$$

- *all three heads are placed at their left-end markers; and*
- *the finite control is in state q_{start}.*

$\Diamond$

The machine proceeds by repeatedly applying its transition function. If it enters q_{halt}, its computation stops. The finite string written immediately to the right of the output tape's left-end marker is interpreted as the machine's output.

[1] Note that the choice of characters is arbitrary. But it is without loss of generality that we only consider binary alphabets, because any finite alphabet can be encoded as a binary string. On potential issue is the size of encoding may grow. But as we will see, this does not affect the computational power of Turing machines, at least to the extent that this course is concerned.

Definition 1.2.3 (Computing a Function and Running Time)**.** *Let $f : \{0,1\}^* \to \{0,1\}^*$, and let $T : \mathbb{N} \to \mathbb{N}$. A Turing machine M* computes *f* in time *$T(n)$ if, for every input $x \in \{0,1\}^*$, the machine:*

1. *starts in the start configuration on x;*
2. *halts after at most $T(|x|)$ transitions; and*
3. *has $f(x)$ written on its output tape when it halts.*

The running time of M is the maximum number of transitions it performs over all inputs of a given length. ◊

The quantity $n = |x|$ is the input length. Complexity theory studies how resource usage grows as a function of n, rather than how long a machine takes on only one particular input.

Definition 1.2.4 (Deciding a Language)**.** *Let $L \subseteq \{0,1\}^*$. A Turing machine M* decides *L if it halts on every input x and outputs*

$$M(x) = \begin{cases} 1, & x \in L, \\ 0, & x \notin L. \end{cases} \tag{1.4}$$

Equivalently, M computes the characteristic function of L. ◊

Remark 1.2.1. *In complexity theory, we normally study machines that halt on every input. A machine that accepts members of a language but may run forever on nonmembers is called a* recognizer *rather than a* decider*. Unless stated otherwise, the algorithms in these notes are deciders.*

Notice that Definition 1.2.4 use the concept of a language to formalize the notion of a decision problem. Do not worry if you are not familiar with the concept of a language. Let us explain it in the following example.

Example 1.2.1 (Representing a Property as a Language)**.** *A language is a set of strings satisfying a specified property. For example, consider the property*

> *"The number of 1's in the binary string is a multiple of 3."*

This property determines the language

$$L_{\text{mult-3}} \coloneqq \{x \in \{0,1\}^* : \textit{the number of } 1\textit{'s in } x \textit{ is a multiple of } 3\}.$$

If $\#_1(x)$ denotes the number of occurrences of 1 in x, then the same language can be written more concisely as

$$L_{\text{mult-3}} = \{x \in \{0,1\}^* : \#_1(x) \equiv 0 \pmod 3\}.$$

For instance,

$$\varepsilon,\ 0,\ 000,\ 111,\ 10101 \in L_{\text{mult-3}},$$

because these strings contain either zero or three occurrences of 1. On the other hand,

$$1,\ 10,\ 110,\ 1111 \notin L_{\text{mult-3}}.$$

Notice that 0 is a multiple of 3, so every string containing no 1's, including the empty string ε, belongs to the language.

This language corresponds to the following decision problem:

Input: A binary string x.

Question: Is the number of 1's in x a multiple of 3?

The answer is "yes" exactly when $x \in L_{\text{mult-3}}$, and it is "no" exactly when $x \notin L_{\text{mult-3}}$.

Remark 1.2.2 (Decision Problems as Languages)**.** *Every decision problem whose instances have finite descriptions can be formalized as a language. First, encode each problem instance I as a binary string $\langle I \rangle$. The language associated with the problem is then*

$$L = \{\langle I \rangle \in \{0,1\}^* : \textit{the answer for instance } I \textit{ is "yes"}\}.$$

Thus, solving the decision problem is equivalent to deciding whether the encoding $\langle I \rangle$ belongs to L.

Strings in L are called yes-instances*, while strings not in L are called* no-instances*.*

Why Turing Machines?

The Turing machine provides a simple but powerful mathematical model of general computation. Its finite control represents the machine's fixed program: the set of states and transition rules is chosen in advance and does not depend on the particular input. The work tape represents memory whose size may grow as the computation proceeds. Thus, unlike a finite automaton, a Turing machine can store an amount of information that is not bounded by a constant.

Despite having potentially unbounded memory, the machine operates locally. During one transition, it can inspect only the three currently scanned symbols, overwrite only the scanned tape cells, and move each head by at most one cell. A complicated computation must therefore be carried out as a sequence of elementary local operations. Consequently, the number of transitions gives a meaningful measure of sequential running time, while the number of tape cells used gives a meaningful measure of memory consumption.

This model is also sufficiently general to express ordinary algorithms. Data structures, intermediate results, and program configurations can all be encoded as strings on the tapes. Although such encodings may be less convenient than the data types and instructions of a programming language, they allow a Turing machine to simulate the essential steps of a computer program. In particular, a universal Turing machine can receive the description of another machine together with its input and simulate that machine, capturing the idea of a stored-program computer.

Finally, the simplicity of the model makes mathematical analysis possible. Questions about whether a problem is computable can be stated precisely in terms of whether some Turing machine decides the corresponding language. Questions about efficiency can likewise be studied by counting the machine's transitions and tape usage. Moreover, standard variants of the model—such as machines with one tape, several tapes, or different tape alphabets—compute the same class of languages and can simulate one another with controlled overhead. The Turing machine therefore abstracts away implementation details while retaining the fundamental notions of algorithm, memory, running time, and computability.

1.3 A Turing Machine for Palindromes

We now use the formal model to design a familiar algorithm. The purpose of this example is not to argue that palindrome testing is difficult. Rather, it illustrates how an ordinary algorithm can be implemented using tapes, head movements, and a finite collection of states.

Definition 1.3.1 (Palindrome Language)**.** *For an alphabet* Σ*, define*

$$\mathrm{PAL}_\Sigma \coloneqq \{x \in \Sigma^* : x = x^R\},$$

where x^R *denotes the reversal of* x*.*

For example,

$$\texttt{radar}^R = \texttt{radar}, \qquad \texttt{complexity}^R = \texttt{ytixelpmoc}.$$

Thus, $\texttt{radar} \in \mathrm{PAL}_\Sigma$*, while* $\texttt{complexity} \notin \mathrm{PAL}_\Sigma$*.* ◊

Updates upto here

Bibliography